AI, ML & Generative AI

Engineering Mental Models, System Architecture & Production Realities

Ihar Rubanovich

Engineering Manager II & AI Ambassador at EPAM Systems. Specialist in systemic AI-Assisted Engineering rollout.



CURRENT ROLES & TRACK RECORD

Engineering Manager II • AI Ambassador @ EPAM

- **10+ Years in IT:** From distributed systems engineering to leading large enterprise delivery teams.
- **AI-Assisted Engineering:** Systemic adoption of autonomous coding agents (CodeMie, Cursor, Antigravity) across production teams.
- **Hands-on AI R&D:** Creator and architect of Creator Tools — a suite of AI tools for YouTube creators designed to save production time, streamline content workflows, and increase earnings.

WORK EMAIL (FOR GITHUB COLLABORATOR INVITATIONS):
ihar_rubanovich@epam.com

Format: Hands-on knowledge transfer grounded in actual enterprise AI deployments.

What You Will Master in GenAI Basics

Six applied engineering modules — from foundational API calls to multimodality, open-source models, and autonomous agents:

MODULE 1 • LESSONS 01-03

Foundations & Python SDK

- L01: AI, ML and generative AI: the big picture
- L02: Tokens, prompts, context, embeddings
- L03: Your first model call from Python (Gemini API)

MODULE 2 • LESSONS 04-06

Multimodal GenAI

- L04: Image generation: services and techniques
- L05: Video generation: models and APIs
- L06: Voice and audio: speech, recognition, music

MODULE 3 • LESSONS 07-09

Ecosystems & Analytics

- L07: The Microsoft Copilot ecosystem
- L08: AI for data analysis
- L09: The Claude ecosystem and artifacts

MODULE 4 • LESSON 10

Open-Source & Local

- L10: Open models: LM Studio, local and RunPod

MODULE 5 • LESSONS 11-13

Agents, Tool Use & MCP

- L11: Function calling and your first agent
- L12: MCP, tools and agent frameworks
- L13: Your own AI assistant

MODULE 6 • LESSONS 14-16

From idea to product

- L14: Rapid prototyping with Streamlit
- L15: Responsible AI, security and regulation
- L16: Agentic development: coding agents

Goal: Transition from casual ChatGPT prompting to architecting and deploying production AI solutions.

Homework Organization: GitHub Repository Standard

All practical assignments are submitted strictly as a structured code repository on GitHub.

genai-homeworks main

```
genai-homeworks/
├── README.md                    # Main index and homework links
│
├── L01/                         # Lesson 01
│   └── README.md                # HW 1: AI Use-Case Memo (.md)
│
├── L02/                         # Lesson 02
│   └── README.md                # HW 2: Token & Prompt Analysis
│
├── L03/                         # Lesson 03
│   ├── solution.py              # Python code (Pydantic / Streamin
│   ├── requirements.txt         # Dependencies
│   └── README.md                # Run instructions
│
└── ...                          # Subsequent modules (L04 - L16)
```

Submission & Review Protocol

1. Create Repository:
Initialize a repository named `genai-homeworks` (Public or Private).

2. Add Instructor to Collaborators:
`Settings` → `Collaborators` → `Add people` :

ihar_rubanovich@epam.com

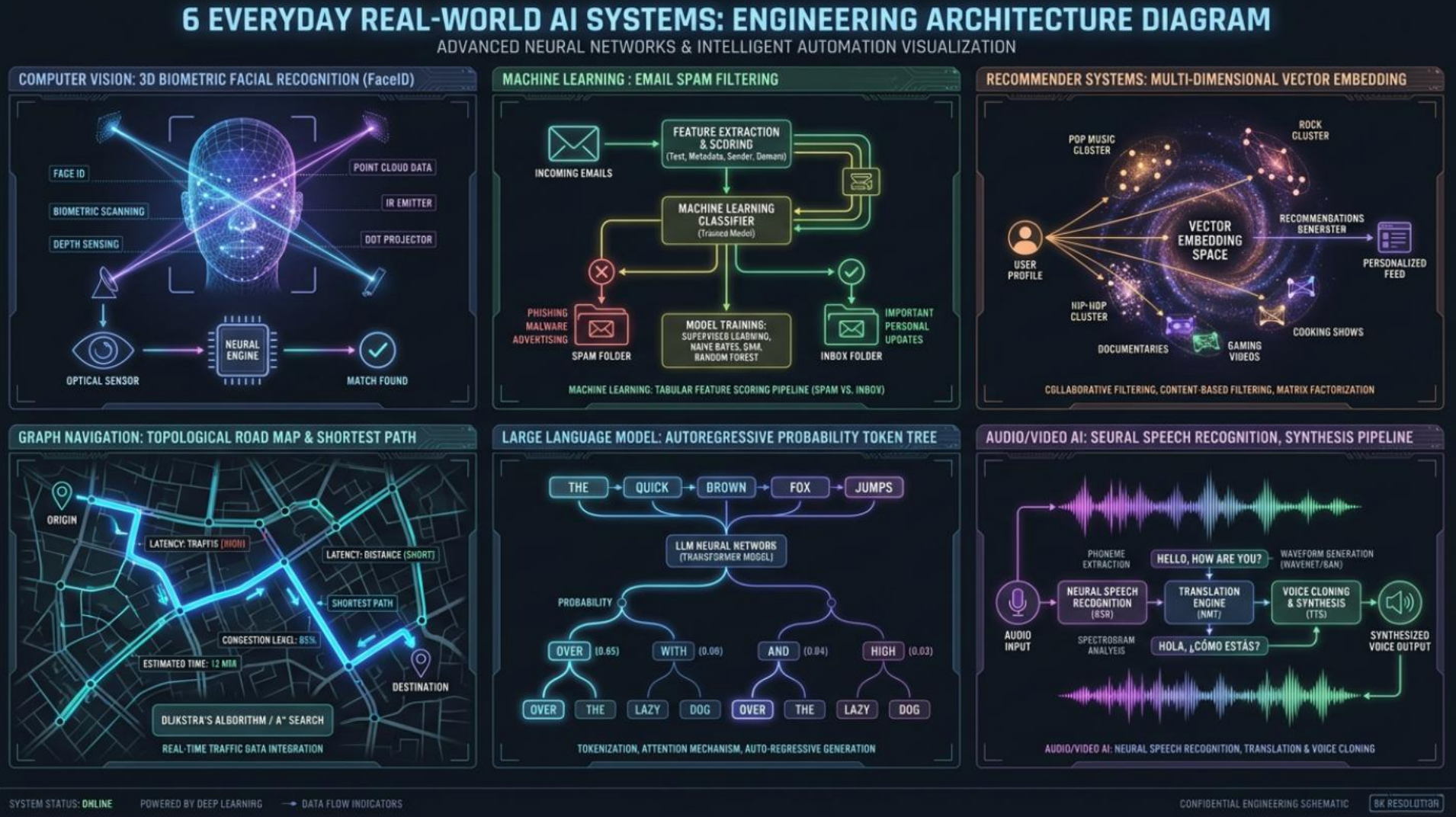
3. Review Process:
Submit the repository link or Pull Request to the learning portal. Each homework receives personal architectural feedback.

Action: Create your repository and invite the collaborator today after lecture.

← / → : Navigate | Space : Next | N : Speaker Notes | G : Grid View | F : Fullscreen

Everyday AI: Technology Operating Around Us

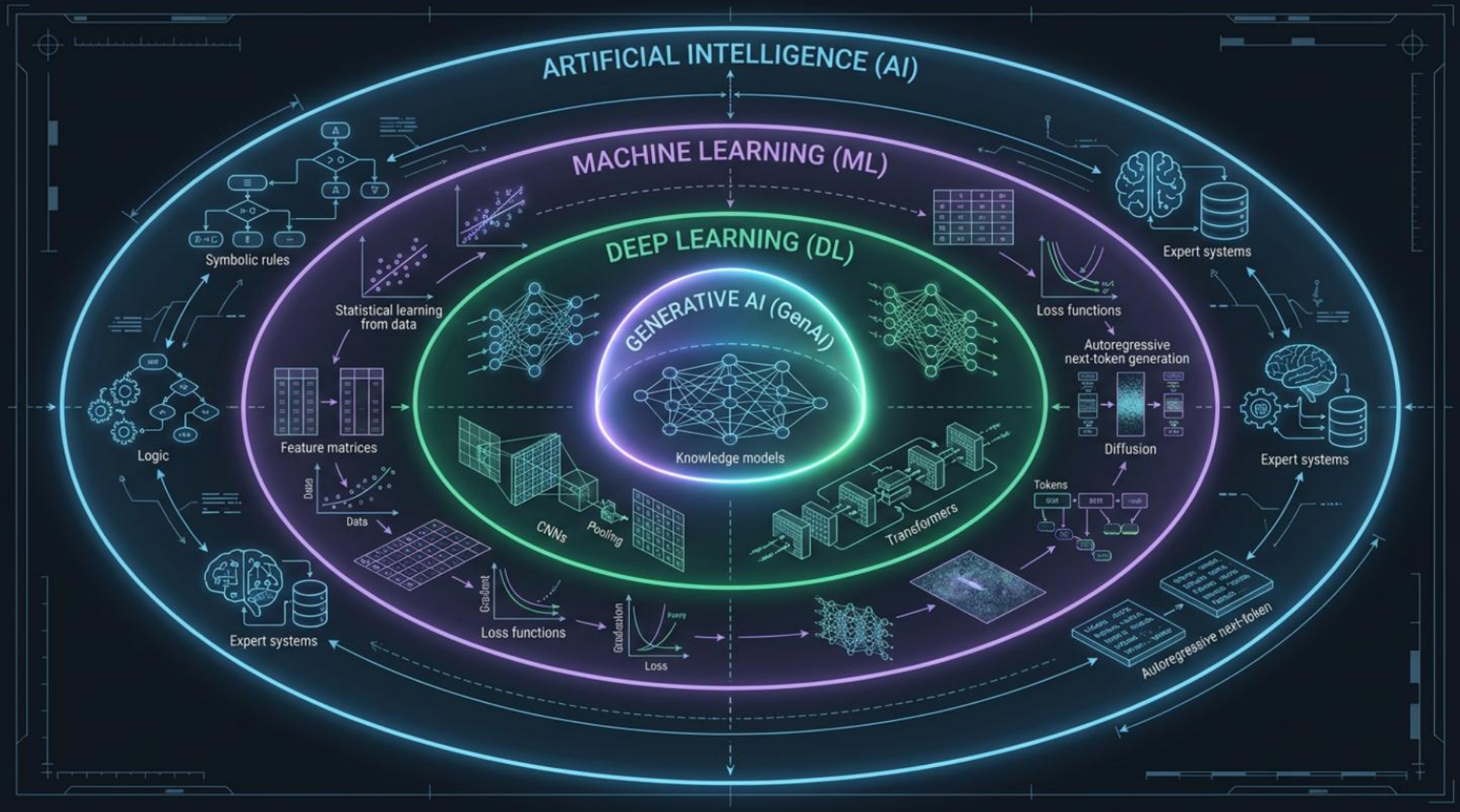
Six real-world production systems — from FaceID and spam classification to vector recommendations and language models:



Takeaway: All these systems are marketed as "AI", yet run on vastly different mathematical foundations under the hood.

Taxonomy: How AI, ML, DL & GenAI Intersect

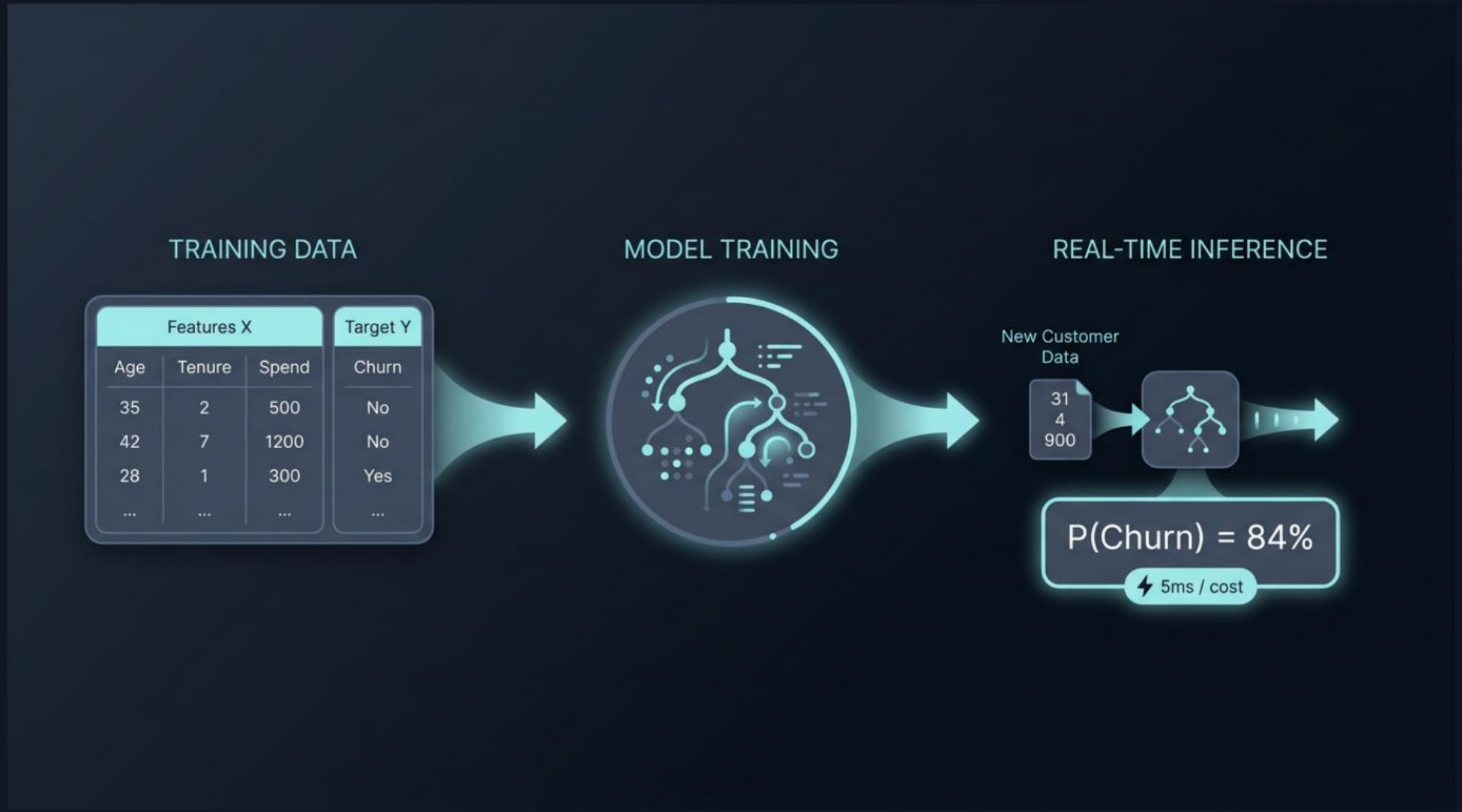
A strict architectural hierarchy: from broad machine intelligence down to modern generative foundation models:



Axiom: Every GenAI model is Deep Learning. But not all Machine Learning is Generative AI.

Classical Machine Learning: Architecture & Properties

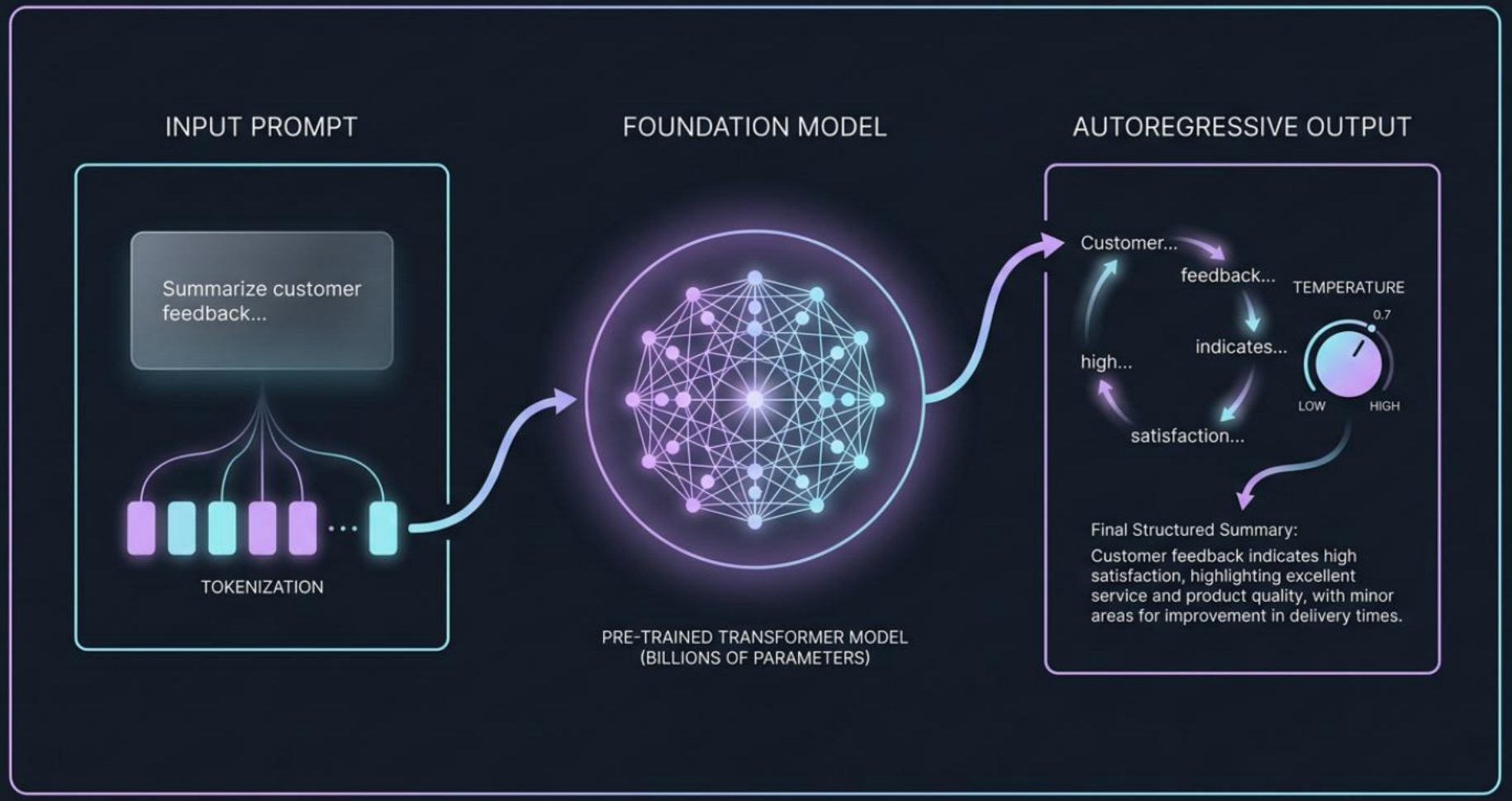
Supervised training on historical tabular records with instant, deterministic probability inference:



ML Advantage: 5ms latency, $0 token cost, calibrated mathematical probabilities on tabular features.

Generative AI: Foundation Model Architecture

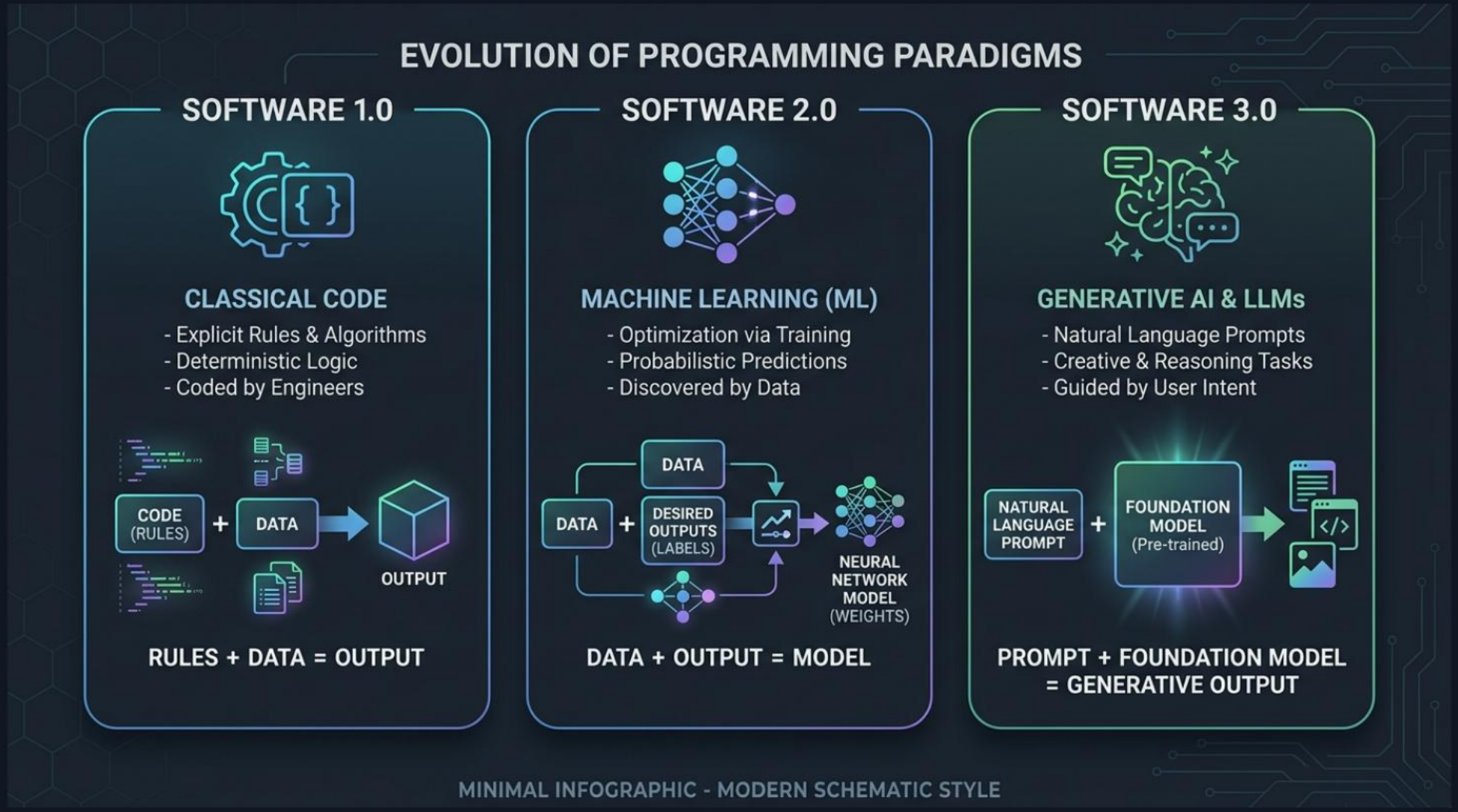
Pre-trained transformer foundation core with autoregressive token-by-token next-state synthesis:



Core Essence: GenAI processes semantics and language, synthesizing answers via probabilistic sampling.

Evolution of Code: Software 1.0 ➔ 2.0 ➔ 3.0

How the paradigm for instructing computers to solve complex problems evolved over the last 20 years:



Role of AI Engineer: Not training models from scratch, but bridging strict 1.0 code with flexible 3.0 foundation models.

The Hype Trap: Why "AI First" Wastes Enterprise Budgets

Forcing an LLM into every problem guarantees high latency, uncontrollable costs, and fragile outcomes:

ANTIPATTERN

“Let's hook an LLM into the accounting ledger!”

- **Latency:** 3 seconds wait time vs 2 milliseconds in SQL.
- **Cost:** $0.03 per invocation vs $0.000001 in conventional code.
- **Accuracy:** 95% vs 100% (5% calculation errors will bankrupt a business).

ENGINEERING MINDSET

“Problem First: Classify the Task Class First”

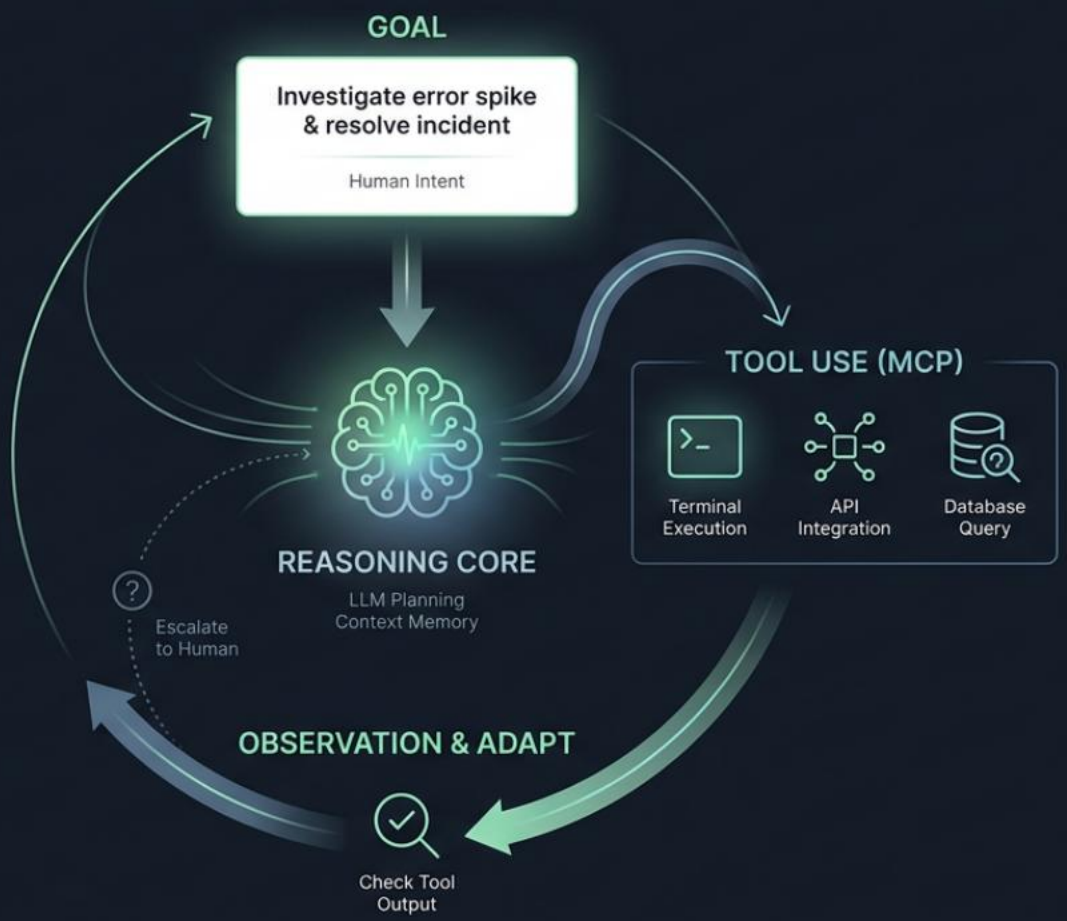
- **Strict formulas & deterministic rules? ➔** Conventional code (100% guarantee).
- **Historical tabular numbers & trends? ➔** Classical ML (speed & statistical rigor).
- **Unstructured text, language & meaning? ➔** Generative AI (governed by human oversight).

Golden Rule: If a problem can be solved in 5 lines of code — calling an LLM is strictly prohibited.

← / → : Navigate | Space : Next | N : Speaker Notes | G : Grid View | F : Fullscreen

Four Paradigms: When to Architect Autonomous AI Agents

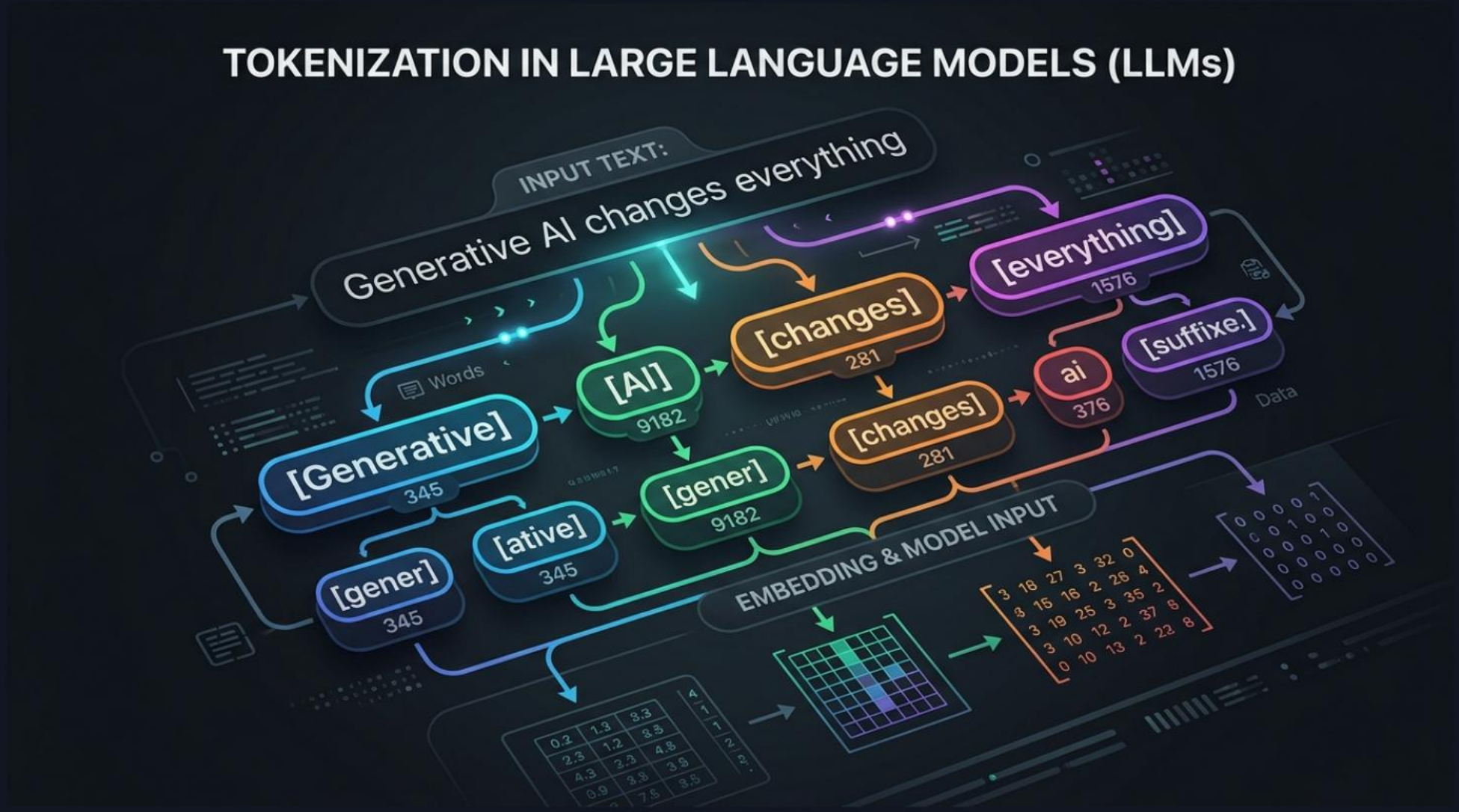
From procedural code and ML to closed-loop perception, multi-step planning, and tool execution (MCP):



Formula: Code calculates. ML predicts. LLM drafts text. An Agent solves open-ended goals in a loop (Goal → Think → Act → Observe).

What Models See: Subword Text Tokenization

Models do not see raw characters or words. They operate on subword tokens and high-dimensional vectors:



Unit Economics: Model providers bill you not for seconds or megabytes, but for exact input and output token counts.

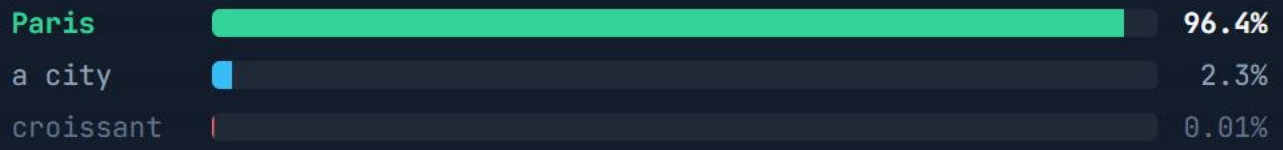
Under the Hood: “T9 on Steroids” & Probabilities

An LLM is an autoregressive calculator for the next token probability distribution:

INPUT CONTEXT (PROMPT):

“The capital of France is [?]”

NEXT TOKEN PROBABILITY DISTRIBUTION:



Temperature Parameter

Temperature = 0.0 (Strict / Greedy):
Always selects the highest-probability token. Ideal for JSON schemas, factual extraction, and code.

Temperature = 0.8 (Creative):
Samples with controlled variability. Ideal for brainstorming, creative copy drafts, and storytelling.

Engineer's Rule: Model output represents statistical co-occurrence patterns in training data, not logical conscious reasoning.

← / → : Navigate | Space : Next | N : Speaker Notes | G : Grid View | F : Fullscreen

Hallucinations vs Grounding: Anchoring Models to Truth

A raw model does not distinguish truth from fiction. It simply completes text plausibly:

AI HALLUCINATION:
Probabilistic Next-Token Guess



SOURCE:
Johnson et al. (2025).
The quantum entanglement
of dream sequences.

VS

GROUNDING (RAG):
Grounded with Verified Document

ANSWER: 100% ACCURATE.
Solar flares result from sudden energy releases in the Sun's atmosphere due to magnetic reconnection.



RAG Principle: Never force a model to recall facts from weights. Inject verified source documents directly into prompt context.

Coffee Break: Focus Refresh

5-minute break. Grab water or coffee and reset your attention before Part 2.

COUNTDOWN TIMER

05:00

START RESET

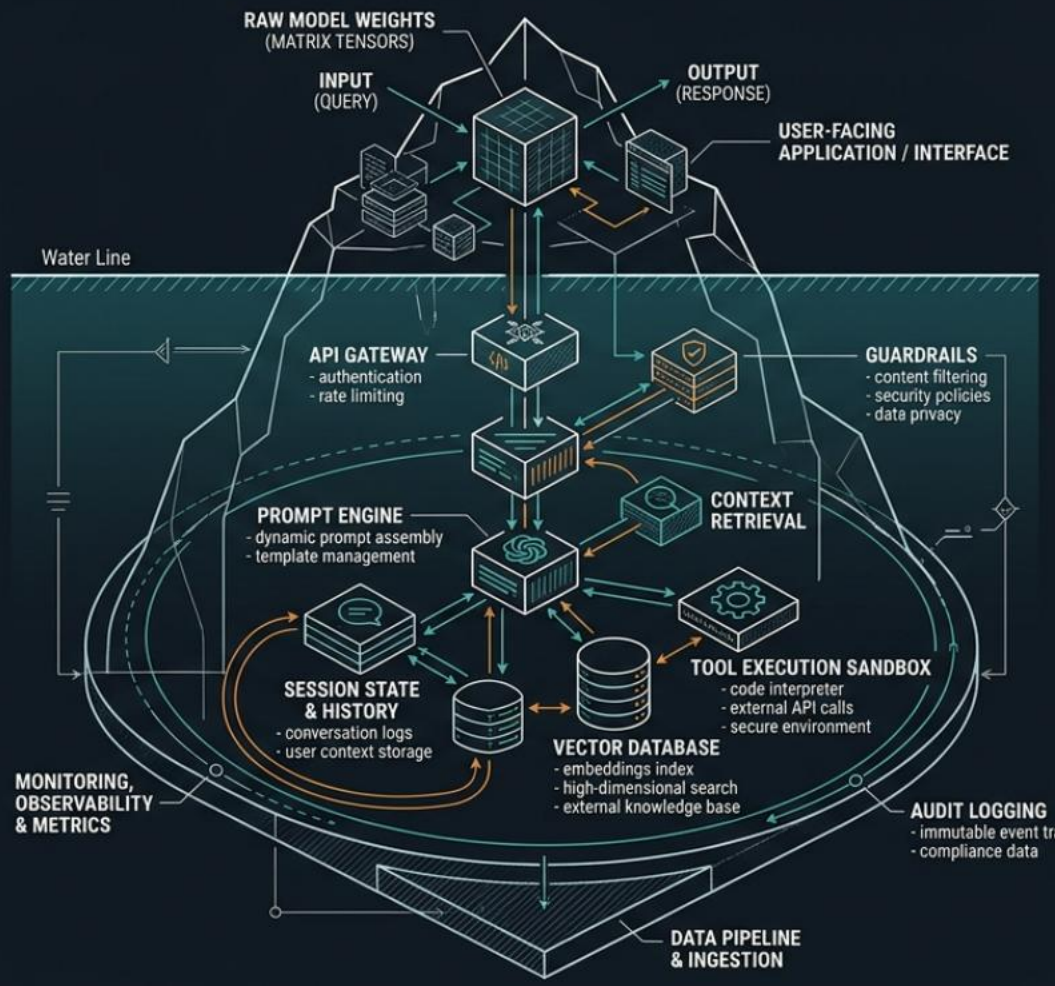
Coming Up in Part 2:

- **The Architectural Iceberg:** why the API call is only 10% of a production system.
- **Blast Radius:** where autonomy works vs. where Human-in-the-Loop is vital.
- **Production Chronicles:** 4 costly industry failures vs. EPAM CodeMie @ Dawn Foods.
- **Decision Tree & Homework #1:** architectural auditing and benchmark walkthrough.

Intermission: Rest for 5 minutes and return for production architecture & homework.

The Architectural Iceberg: Model ≠ Product

Invoking the model API takes 10 lines of code. Building a dependable production system requires the remaining 90% of infrastructure:



Engineering Truth: A demo is hacked together in an evening. Production takes months of engineering around resilience, retries, and safety.

Blast Radius Spectrum & Human-in-the-Loop

Model autonomy must be strictly inversely proportional to the cost and damage of an error:

LOW RISK • GREEN ZONE

Drafts & Creativity

- Summarizing lengthy articles
- Brainstorming ideas & email drafts
- Internal documentation search

Autonomy: Full. A hallucination costs $0 and harms no one.

MEDIUM RISK • YELLOW ZONE

Coding & Data Preparation

- IDE code autocompletion (Copilot)
- Initial Pull Request review
- Customer ticket categorization

Autonomy: Co-pilot. Human always verifies before merging.

HIGH RISK • RED ZONE

Finance, Prod & Legal

- Processing payments & invoices
- Legal contracts & binding terms
- Autonomous production deployments

Autonomy: Prohibited. Model only prepares advisory drafts.

Safety Principle: The higher the cost of failure, the stricter the Human-in-the-Loop gate.

← / → : Navigate | Space : Next | N : Speaker Notes | G : Grid View | F : Fullscreen

Industry Case Studies: 4 Costly Fails vs 4 Solid Wins

Hard lessons from enterprise deployments: where companies burned millions vs where they achieved lasting ROI:

4 COSTLY FAILS (UNDERESTIMATING RISK)

- **Air Canada ($):** Support chatbot promised a bereavement discount contrary to policy. Tribunal ruled the airline legally liable for damages.
- **Chevrolet for $1:** Dealership bot agreed to sell a customer a Tahoe for $1 due to a trivial prompt injection jailbreak.
- **DPD Logistics:** Delivery chatbot swore at a customer and composed a poem describing the company's uselessness.
- **NYC Business Bot:** Official municipal AI consultant actively advised entrepreneurs to violate local labor laws.

4 SOLID WINS (SYSTEMIC ROI)

- **EPAM CodeMie @ Dawn Foods:** SDLC AI orchestrator integration: 35% feature delivery and test generation acceleration with preserved architectural context.
- **GitHub Copilot:** 55% acceleration in boilerplate coding and unit test writing across hundreds of thousands of engineers worldwide.
- **Stripe Fraud Detection:** Hybrid ensemble combining classical ML scoring with LLMs for complex financial fraud investigations.
- **Morgan Stanley:** Internal grounded RAG-GPT for instantaneous search across 100,000 proprietary equity research reports.

Takeaway: Winners anchor models to verified knowledge bases and enforce strict output validation layers.

Engineering Decision Tree: Do You Actually Need GenAI?

An algorithmic filter before writing project code. Route every user requirement through 4 questions:

01	Can the task be solved by deterministic algorithms, regex, or structured SQL?	YES ➔ Write conventional code
02	Does the problem require numeric predictions on historical tabular data (churn, demand, prices)?	YES ➔ Train classical ML
03	Does the task involve unstructured text, semantic understanding, code, or creative synthesis?	YES ➔ Apply Generative AI
04	Is 100% accuracy mission-critical, and is a Human-in-the-Loop review gate architected?	Validation plan required

Engineering Maturity: Do not solve problems with a sledgehammer when facing a screw.

Homework: AI Use-Case Memo

Bridging theory to architectural audit: assignment requirements and a concrete production example:

Memo Requirements (300-500 words):

1. Process Description: Select 1 real-world workflow from your current work.

2. Decision Tree Classification: Justify why GenAI is needed rather than SQL or ML.

3. Blast Radius Assessment: Define risk tier (Green / Yellow / Red) and autonomy degree.

4. Architectural Taboo: What action is the model strictly prohibited from performing?

GITHUB SUBMISSION PROTOCOL:

File: `genai-homeworks/L01/README.md`

Collaborator: `ihar_rubanovich@epam.com`

Submission Sample: Glovo Food Delivery Courier Case

HW SAMPLE

1. Process: Courier arrives at restaurant, but ordered dessert is out of stock (or gate code missing). Courier needs urgent customer communication without delaying delivery.

2. Decision Tree:

- *Route & ETA:* Road graph + ML traffic model (100% math, 5ms latency).
- *Customer Chat:* **GenAI** (polite adaptive message in customer's native language).

3. Blast Radius (Yellow Zone • Co-pilot): Model generates ready draft ("Hello! Cheesecake is out of stock, restaurant offers tiramisu at no cost. Swap?"), but **courier confirms send with 1 tap**.

4. Architectural Taboo: Model is **strictly forbidden** from: cancelling orders, issuing refunds, offering platform-funded promo codes, or assessing restaurant fines.

Deadline: Submit before next lecture. Every memo receives individual architectural feedback.

← / → : Navigate | Space : Next | N : Speaker Notes | G : Grid View | F : Fullscreen

Q&A Session & Preview of Lesson 02

Live discussion of your production use cases and introduction to deep model mechanics:

Open Microphone

Ask questions in the chat or speak up on voice.

Any dilemmas regarding choosing between deterministic code, classical ML, and LLMs in your teams are welcome!

UPCOMING SESSION • LESSON 02

LLM Fundamentals: Tokens, Context & RAG

- Deep dive into BPE tokenization implemented in Python.
- Context window physics: why 128k does not prevent lost-in-the-middle degradation.
- Vector embeddings and building your first grounded RAG pipeline.

Contact: ihar_rubanovich@epam.com • Session materials available in the GitHub repository.

← / → : Navigate | Space : Next | N : Speaker Notes | G : Grid View | F : Fullscreen